MODELO DE CLASES UML PARA UN SISTEMA DE OPERACIONES ARITMÉTICAS CON AUTENTICACIÓN DE USUARIOS

PRESENTADO POR:
ALEJANDRO DE MENDOZA

POLITECNICO GRANCOLOMBIANO
MAESTRÍA ARQUITECTURA DE SOFTWARE
TEMAS AVANZADOS DE DISEÑO SOFTWARE

PRESENTADO AL PROFESOR:
ING. LINA MARIA MONTOYA SUAREZ

23 DE MARZO
BOGOTÁ D.C.
2026

TABLA DE CONTENIDO

INTRODUCCIÓN ____ ***4***

DESARROLLO DEL PROYECTO ____ ***4***

Parte 1. Arquitectura Del Proyecto ____ **5**

Estructura general del proyecto ____ 6

1. Carpeta components/ ____ 6
2. Carpeta clients/ ____ 6
3. Carpeta public/ ____ 6
4. Archivo db.js ____ 6
5. Archivos de configuración ____ 7
6. Enfoque arquitectónico ____ 7
7. Flujo general del sistema ____ 7

Parte 2. Clases Del Sistema ____ **7**

1. Vendedor ____ 7
2. RecursosHumanos ____ 7
3. Cliente ____ 8
4. Producto ____ 8
5. Venta ____ 8
6. Bodega ____ 8
7. Proveedor ____ 8
8. Entrega ____ 8

Relaciones del modelo ____ 9

Asociaciones ____ 9

Dependencias ____ 9

Parte 3. Código Y Diagrama De Clases Del Sistema Polimarket ____ **9**

1. Código: ____ 9
2. Imagen del diagrama en PlantUML ____ 11
3. Imagen del diagrama en Visual Code ____ 11

Parte 4. Componentes Del Sistema ____ **12**

1. Tabla de definición de componentes ____ 12
2. ComponenteAutorizacion (Recursos Humanos) ____ 13

   Funcionalidades: ____ 13

   Relación con el sistema: ____ 13
3. ComponenteRegistroVentas (Ventas) ____ 13

   Funcionalidades: ____ 13

   Relación con el sistema: ____ 13
4. ComponenteInventario (Bodega) ____ 14

   Funcionalidades: ____ 14

   Relación con el sistema: ____ 14
5. ComponenteReabastecimiento (Bodega / Proveedores) ____ 14

   Funcionalidades: ____ 14

   Relación con el sistema: ____ 14
6. ComponenteGestionEntregas (Entregas) ____ 14

   Funcionalidades: ____ 14

   Relación con el sistema: ____ 15

   Integración de componentes ____ 15

Parte 5. Implementación De Requisitos Funcionales ____ **15**

Estructura del proyecto ____ 15

RF1 - Autorizar vendedor ____ 16

Descripción: ____ 16

Componente asociado: 16
Funcionamiento: 16
Importancia: 16
RF2 - Registrar venta 16
Descripción: 16
Componentes asociados: 16
Funcionamiento: 16
Importancia: 16
RF3 - Verificar disponibilidad de productos 17
Descripción: 17
Componente asociado: 17
Funcionamiento: 17
Importancia: 17
RF4 - Solicitar reabastecimiento 17
Descripción: 17
Componente asociado: 17
Funcionamiento: 17
Importancia: 17
RF5 - Registrar entrega 17
Descripción: 18
Componentes asociados: 18
Funcionamiento: 18
Importancia: 18
Integración con los clientes 18

Parte 6. Diagrama de componentes del sistema PoliMarket 18
1. Código: 18
2. Imagen del diagrama en PlantUML 20
3. Imagen del diagrama en Visual Code 20

Parte 7. Clientes implementados 21
1. Cliente 1 - API REST con Express (plataforma web) 21
2. Cliente 2 - CLI interactivo (plataforma terminal) 21
3. Tabla de funcionalidades por cliente 21

Parte 8. Instrucciones de ejecución del programa 21

Parte 9. Imagen Del Desarrollo 22

Parte 10. Archivos Adjuntos En Adición 22

CONCLUSIONES DEL DESARROLLO 22

BIBLIOGRAFÍA 23

AGRADECIMIENTO 24

INTRODUCCIÓN

En el contexto actual del desarrollo de software, el modelado de sistemas se ha consolidado como una etapa fundamental para garantizar soluciones estructuradas, escalables y de alta calidad. A través del uso del Lenguaje Unificado de Modelado (UML), es posible representar de manera clara y precisa la arquitectura de un sistema antes de su implementación, facilitando la comprensión de sus componentes, sus responsabilidades y las interacciones entre ellos.

En este sentido, el enfoque orientado a objetos permite identificar las entidades principales del sistema, definir sus atributos y métodos, y establecer las relaciones que existen entre ellas. Este enfoque promueve buenas prácticas como la modularidad, el bajo acoplamiento y la alta cohesión, elementos clave para el desarrollo de aplicaciones mantenibles y extensibles, especialmente en entornos empresariales donde intervienen múltiples áreas de negocio.

A partir de este enfoque, se plantea el diseño de un sistema de información para la empresa PoliMarket, el cual integra diferentes áreas organizacionales como recursos humanos, ventas, bodega, proveedores y entregas. El modelo considera la interacción entre estas áreas, permitiendo gestionar procesos como la autorización de vendedores, el registro de ventas, la consulta de disponibilidad de productos, el reabastecimiento de inventario y la gestión de entregas.

Adicionalmente, el sistema incorpora una arquitectura basada en componentes, donde cada área de negocio expone funcionalidades específicas a través de módulos reutilizables. Este enfoque permite separar la lógica del sistema en unidades independientes, facilitando su mantenimiento, escalabilidad y reutilización en diferentes contextos.

Asimismo, se contempla la interacción del sistema mediante múltiples clientes, incluyendo una API REST para consumo web y una interfaz de línea de comandos (CLI), lo que evidencia la capacidad de reutilizar la lógica de negocio en distintas plataformas sin necesidad de duplicar código.

Finalmente, mediante la construcción del modelo de clases UML y el modelo de componentes, se busca representar de manera estructurada los elementos del sistema, incluyendo clases, atributos, métodos y relaciones, así como la interacción entre los diferentes módulos. Esto permite facilitar su análisis, comprensión y posterior implementación, evidenciando la importancia del modelado como base para el desarrollo de software de calidad.

DESARROLLO DEL PROYECTO

A continuación, se presenta el desarrollo estructurado de la actividad “Reutilización de software”, orientada al análisis, diseño e implementación de un sistema de información mediante el uso del Lenguaje Unificado de Modelado (UML) y el enfoque basado en componentes. El propósito de este ejercicio es aplicar los conceptos fundamentales del diseño orientado a objetos y la reutilización de software, con el fin de construir una solución que represente de manera clara, organizada y coherente la estructura del sistema propuesto.

El desarrollo se organiza en secciones que abordan los elementos esenciales del modelado y la arquitectura de software. En una primera parte, se identifican los conceptos clave

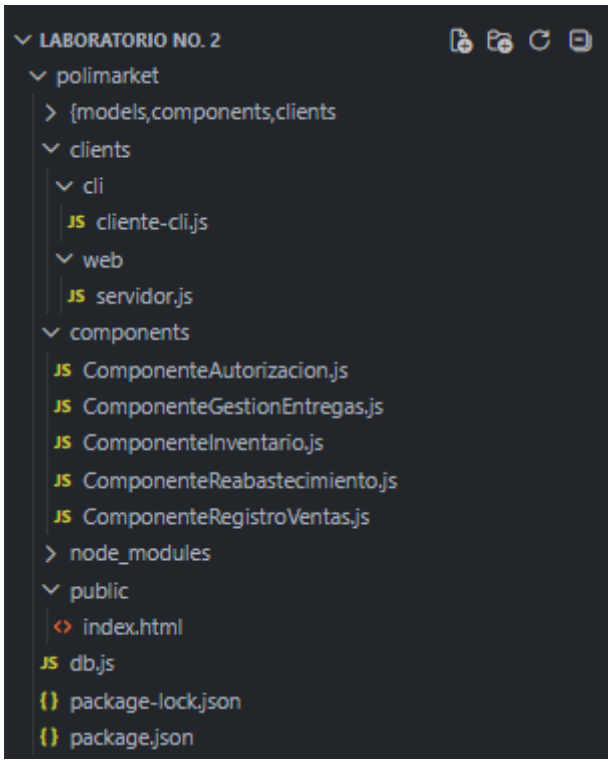
que pueden ser representados como clases dentro del sistema PoliMarket, justificando su inclusión a partir de los requerimientos planteados. Posteriormente, se establecen las relaciones entre dichas clases, incorporando principios como la asociación, la dependencia y la multiplicidad, los cuales permiten estructurar de forma precisa las interacciones del sistema.

De manera complementaria, se identifican los componentes de software asociados a cada área de negocio -recursos humanos, ventas, bodega, proveedores y entregas- definiendo las funcionalidades que cada uno expone. Esto permite evidenciar el uso de la reutilización de software mediante la construcción de módulos independientes que pueden ser consumidos por diferentes clientes.

Asimismo, el modelo contempla la interacción del sistema a través de múltiples clientes, específicamente una API REST y una interfaz de línea de comandos (CLI), lo que permite desacoplar la lógica del negocio de los mecanismos de acceso al sistema y facilita la reutilización de los componentes desarrollados.

En las siguientes secciones se detallan las clases propuestas, sus atributos y métodos, la definición de los componentes y sus funcionalidades, así como la representación del sistema mediante diagramas UML de clases y de componentes. Finalmente, se presenta la implementación de los requisitos funcionales, evidenciando cómo el diseño propuesto se traduce en una solución funcional y reutilizable.

Parte 1. Arquitectura Del Proyecto



El sistema desarrollado para PoliMarket sigue una arquitectura modular basada en componentes, estructurada de manera que se facilite la separación de responsabilidades, la reutilización de código y la escalabilidad de la solución. La organización del proyecto permite diferenciar claramente las capas de acceso, lógica de negocio y almacenamiento de datos.

La estructura del proyecto se compone de los siguientes elementos principales:

Estructura general del proyecto

El sistema se organiza en carpetas que representan las distintas capas funcionales:

1. Carpeta components/

Contiene los componentes de negocio del sistema, los cuales encapsulan la lógica principal de cada área organizacional. Cada archivo representa un componente reutilizable que expone funcionalidades específicas:

- ComponenteAutorizacion.js → Gestión de autorización de vendedores (Recursos Humanos)
- ComponenteRegistroVentas.js → Registro y gestión de ventas (Ventas)
- ComponenteInventario.js → Control de stock y movimientos (Bodega)
- ComponenteReabastecimiento.js → Solicitud de productos a proveedores (Bodega/Proveedores)
- ComponenteGestionEntregas.js → Gestión de entregas y despachos (Entregas)

Estos componentes constituyen la capa de lógica de negocio, permitiendo que las funcionalidades puedan ser reutilizadas por diferentes clientes sin duplicar código.

2. Carpeta clients/

Contiene las implementaciones de los clientes que consumen los componentes del sistema:

- web/servidor.js → Cliente web basado en API REST utilizando Express
- cli/cliente-cli.js → Cliente de línea de comandos (CLI)

Estos clientes representan la capa de acceso al sistema, permitiendo interactuar con las funcionalidades de negocio desde diferentes plataformas, lo cual evidencia la reutilización de los componentes.

3. Carpeta public/

Incluye archivos estáticos para el cliente web:

- index.html → Interfaz básica de usuario para consumo de la API

4. Archivo db.js

Este archivo simula una base de datos en memoria, donde se almacenan entidades como vendedores, productos, clientes y ventas. Permite desacoplar la lógica de negocio de la persistencia de datos, facilitando futuras integraciones con bases de datos reales.

5. Archivos de configuración

- package.json → Define dependencias y scripts del proyecto
- package-lock.json → Control de versiones de dependencias

6. Enfoque arquitectónico

La arquitectura implementada sigue un enfoque basado en:

- Separación de responsabilidades: cada componente tiene una función específica dentro del sistema.
- Reutilización de software: los componentes pueden ser consumidos por múltiples clientes sin modificación.
- Bajo acoplamiento: los clientes no dependen directamente de la lógica interna, sino de las funcionalidades expuestas por los componentes.
- Escalabilidad: la estructura permite agregar nuevos componentes o clientes sin afectar los existentes.

7. Flujo general del sistema

- El cliente (web o CLI) envía una solicitud.
- La solicitud es procesada por el componente correspondiente.
- El componente interactúa con la base de datos (db.js).
- Se retorna la respuesta al cliente.

Parte 2. Clases Del Sistema

A partir del análisis del problema planteado para la empresa PoliMarket, se identificaron las siguientes clases como elementos fundamentales del sistema, las cuales permiten representar de manera estructurada las entidades del dominio y sus responsabilidades dentro de la organización:

1. Vendedor

Representa al empleado encargado de realizar las ventas dentro del sistema. Esta clase es fundamental, ya que es el principal actor en el proceso comercial. Requiere autorización previa del área de Recursos Humanos para poder interactuar con los demás componentes del sistema.

- Atributos: id, nombre, email, autorizado
- Métodos: solicitarAutorizacion(), realizarVenta(), consultarClientes(), consultarProductos()

2. RecursosHumanos

Gestiona la autorización y control de acceso de los vendedores al sistema. Su inclusión es esencial, ya que permite garantizar que solo usuarios habilitados puedan ejecutar operaciones dentro de la organización.

- Atributos: id, responsable
- Métodos: autorizarVendedor(), revocarAutorizacion(), listarVendedores()

3. Cliente

Representa a la persona que realiza compras en PoliMarket. Esta clase permite almacenar la información del cliente y consultar su historial de compras, siendo clave en el proceso de ventas.

- Atributos: id, nombre, email, direccion
- Métodos: getDatos(), getHistorialCompras()

4. Producto

Representa los artículos disponibles para la venta. Su estado es gestionado por la bodega, especialmente en lo relacionado con la disponibilidad y actualización del inventario.

- Atributos: id, nombre, precio, stock
- Métodos: getDisponibilidad(), actualizarStock()

5. Venta

Representa una transacción comercial realizada en el sistema. Esta clase vincula al vendedor con el cliente y los productos adquiridos, siendo el núcleo del proceso de negocio.

- Atributos: id, fecha, total, estado
- Métodos: registrarVenta(), calcularTotal(), getDetalle()

6. Bodega

Gestiona el inventario físico de productos dentro de la organización. Interactúa con las áreas de ventas, proveedores y entregas para controlar el flujo de entrada y salida de productos.

- Atributos: id, ubicacion, capacidad
- Métodos: verificarDisponibilidad(), registrarEntrada(), registrarSalida(), solicitarReabastecimiento()

7. Proveedor

Representa las empresas externas encargadas de suministrar productos a PoliMarket. Permite gestionar el abastecimiento del inventario cuando los niveles de stock son bajos.

- Atributos: id, nombre, contacto, telefono
- Métodos: registrarProducto(), getProductosSuministrados(), solicitarPedido()

8. Entrega

Gestiona la distribución de los productos vendidos hacia los clientes. Se comunica con el área de ventas para conocer los pedidos y con la bodega para registrar la salida de productos.

- Atributos: id, fecha, estado, direccionDestino
- Métodos: registrarEntrega(), consultarPedidosPendientes(), confirmarEntrega(), registrarSalidaBodega()

Relaciones del modelo

Las relaciones entre las clases permiten representar la interacción entre los diferentes elementos del sistema, de acuerdo con los procesos de negocio de PoliMarket:

Asociaciones

- RecursosHumanos (1) → (0..*) Vendedor: Indica que un área de recursos humanos puede autorizar múltiples vendedores.
- Vendedor (1) → (0..*) Venta: Un vendedor puede realizar múltiples ventas, pero cada venta es realizada por un único vendedor.
- Venta (0..*) → (1) Cliente: Cada venta pertenece a un único cliente, mientras que un cliente puede tener múltiples compras.
- Venta (1..) → (1..) Producto: Una venta incluye uno o varios productos, y un producto puede estar presente en múltiples ventas.
- Bodega (1) → (0..*) Proveedor: La bodega se relaciona con múltiples proveedores para el abastecimiento de productos.

Dependencias

- Venta → Bodega: La venta depende de la bodega para verificar la disponibilidad de los productos antes de ser registrada.
- Entrega → Venta: La entrega depende de la información de las ventas para conocer qué productos deben ser despachados.
- Entrega → Bodega: La entrega interactúa con la bodega para registrar la salida de productos del inventario.

En conjunto, las clases y relaciones definidas permiten representar de manera estructurada los procesos de negocio de PoliMarket, evidenciando la aplicación del modelado orientado a objetos para la construcción de sistemas organizacionales complejos.

Parte 3. Código Y Diagrama De Clases Del Sistema Polimarket

A continuación, se presenta tanto el código como el diagrama de las clases identificadas para el sistema PoliMarket, junto con sus atributos, métodos y justificación de inclusión. El diagrama fue generado con PlantUML usando la sintaxis UML 2.5.

1. Código:

File Edit Selection View Go Run Terminal Help

EXPLORER

LABORATORIO NO. 2

- polimarket
  - {models,components,clients
  - clients
    - cli
      - cliente-cli.js
    - web
      - servidor.js
  - components
    - ComponenteAutorizacion.js
    - ComponenteGestionEntregas.js
    - ComponenteInventario.js
    - ComponenteReabastecimiento.js
    - ComponenteRegistroVentas.js
  - node_modules
  - public
  - uml
    - diagrama_clases.bmp
    - diagrama-clases.puml
    - diagrama-componentes.BMP
    - diagrama-componentes.puml
  - db.js
  - package-lock.json
  - package.json
- ~$liMarket_Unidad2_Alejandro_De_Mendoza_Final.docx
- ~$sarrollo Proyecto Alejandro De Mendoza.docx
- ~$tividad Evaluativa_Sumativa_U2.docx
- ~WRL1168.tmp
- Actividad Evaluativa_Sumativa_U2.docx
- Actividad Evaluativa_Sumativa_U2.pdf
- Desarrollo Proyecto Alejandro De Mendoza.docx
- PoliMarket_Unidad2_Alejandro_De_Mendoza_Final.docx
- PoliMarket_Unidad2_Alejandro_De_Mendoza_Final.pdf

diagrama-clases.puml

polimarket > uml > diagrama-clases.puml > { } PoliMarket_Clases

```
@startuml PoliMarket_Clases
skinparam shadowing false
skinparam linetype ortho

class RecursosHumanos {
  - id : String
  - responsable : String
  + autorizarVendedor(vendedorId: String) : boolean
  + revocarAutorizacion(vendedorId: String) : void
  + listarVendedores() : List<Vendedor>
}

class Vendedor {
  - id : String
  - nombre : String
  - email : String
  - autorizado : boolean
  + solicitarAutorizacion() : void
  + realizarVenta(clienteId: String) : Venta
  + consultarClientes() : List<Cliente>
  + consultarProductos() : List<Producto>
}

class Cliente {
  - id : String
  - nombre : String
  - email : String
  - direccion : String
  + getDatos() : Map<String, String>
  + getHistorialCompras() : List<Venta>
}

class Producto {
  - id : String
  - nombre : String
  - precio : double
  - stock : int
  + getDisponibilidad() : int
  + actualizarStock(cantidad: int) : void
}

class Venta {
  - id : String
  - fecha : Date
  - total : double
  - estado : String
  + registrarVenta() : void
  + calcularTotal() : double
  + getDetalle() : Map<String, Object>
}

class Bodega {
  - id : String
  - ubicacion : String
  - capacidad : int
  + verificarDisponibilidad(productoId: String) : int
  + registrarEntrada(productoId: String, cantidad: int) : void
  + registrarSalida(productoId: String, cantidad: int) : void
  + solicitarReabastecimiento(productoId: String) : void
}

class Proveedor {
  - id : String
  - nombre : String
  - contacto : String
  - telefono : String
  + registrarProducto(producto: Producto) : void
  + getProductosSuministrados() : List<Producto>
  + solicitarPedido(productoId: String, cantidad: int) : void
}

class Entrega {
  - id : String
  - fecha : Date
  - estado : String
  - direccionDestino : String
  + registrarEntrega() : void
  + consultarPedidosPendientes() : List<Venta>
  + confirmarEntrega(entregaId: String) : void
  + registrarSalidaBodega(productoId: String, cantidad: int) : void
}

RecursosHumanos "1" --> "0..*" Vendedor : autoriza >
Vendedor "1" --> "0..*" Venta : realiza >
Venta "0..*" --> "1" Cliente : pertenece a >
Venta "1..*" --> "1..*" Producto : incluye >
Venta ..> Bodega : <<consulta>>
Bodega "1" --> "0..*" Proveedor : registra >
Entrega ..> Venta : <<consulta>>
Entrega ..> Bodega : <<registra salida>>

@enduml
```

2. Imagen del diagrama en PlantUML

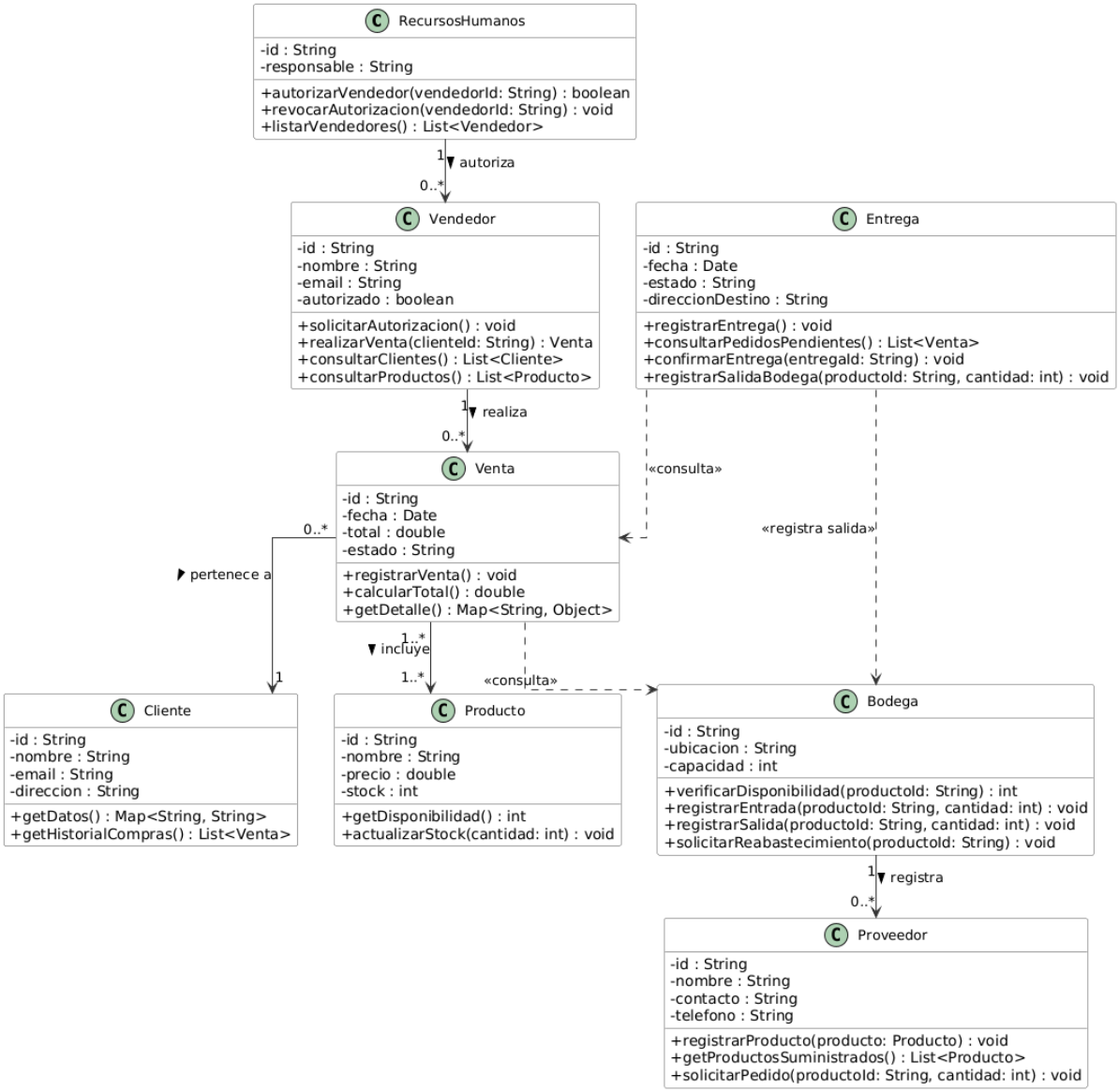


Figura 1. Diagrama de clases del sistema PoliMarket

La imagen anterior corresponde al diagrama de clases del sistema PoliMarket, exportado en formato BMP desde la herramienta PlantUML (plantuml.com) utilizando la sintaxis UML 2.5. En la entrega en el código de igual manera en la carpeta de nombre “uml” se adjunta esta imagen bajo el nombre diagrama-clases.bmp. El diagrama evidencia las ocho clases identificadas, sus atributos, métodos, las relaciones de asociación y dependencia con sus respectivas multiplicidades.

3. Imagen del diagrama en Visual Code

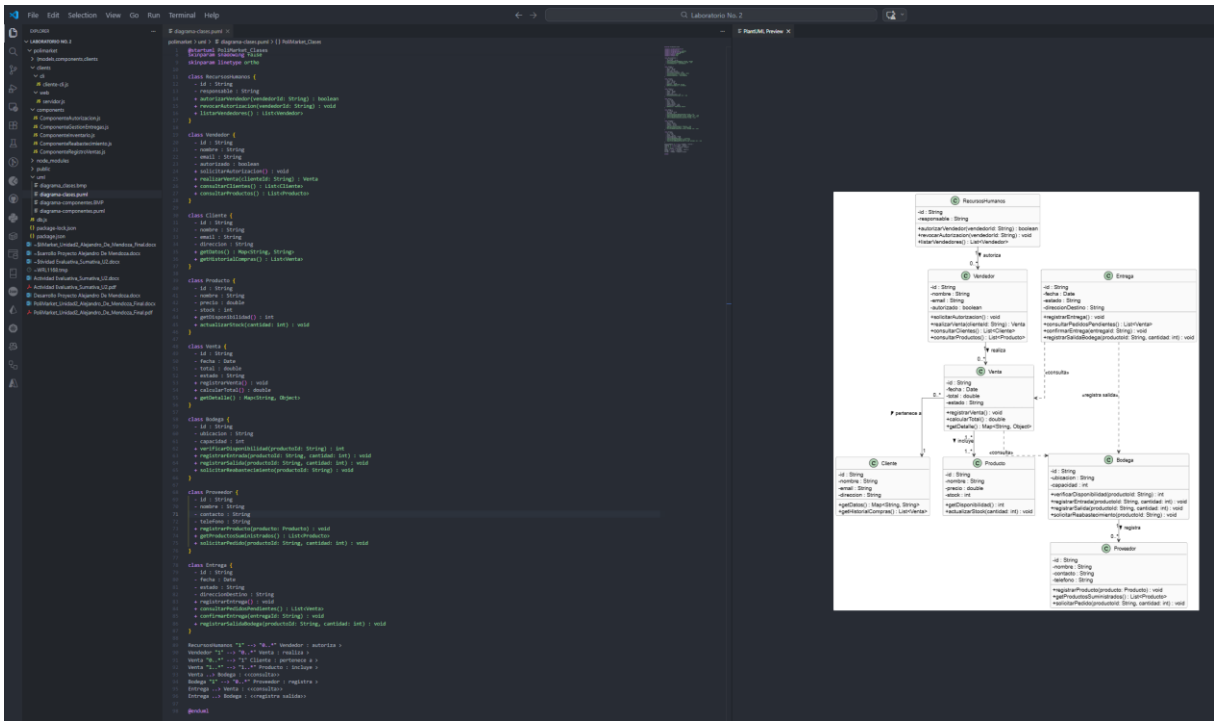


Figura 3. Diagrama de clases del sistema PoliMarket en Visual Code

Parte 4. Componentes Del Sistema

El sistema PoliMarket se estructura mediante componentes de software que encapsulan la lógica de negocio de cada área organizacional. Cada componente expone funcionalidades específicas que pueden ser reutilizadas por diferentes clientes, permitiendo una arquitectura modular, flexible y escalable.

A continuación, se describen los principales componentes implementados:

1. Tabla de definición de componentes

A continuación, se presenta la Tabla 1 con los componentes identificados para cada área de negocio de PoliMarket. Se definieron mínimo dos componentes por área, especificando las funcionalidades que cada uno expone.

Área de negocio	Componente	Funcionalidades que expone
Recursos Humanos	ComponenteAutorizacion	autorizarVendedor(), revocarAutorizacion()
Recursos Humanos	ComponenteGestionVendedores	listarVendedores(), consultarEstadoVendedor()
Ventas	ComponenteRegistroVentas	registrarVenta(), calcularTotal(), getDetalle()
Ventas	ComponenteConsultaVentas	listarClientes(), listarProductosDisponibles()

Área de negocio	Componente	Funcionalidades que expone
Bodega	ComponenteInventario	verificarDisponibilidad(), registrarEntrada(), registrarSalida()
Bodega	ComponenteReabastecimiento	solicitarReabastecimiento(), consultarProveedores()
Proveedores	ComponenteGestionProveedores	registrarProducto(), getProductosSuministrados()
Proveedores	ComponentePedidos	solicitarPedido(), consultarEstadoPedido()
Entregas	ComponenteGestionEntregas	registrarEntrega(), confirmarEntrega()
Entregas	ComponenteLogistica	consultarPedidosPendientes(), registrarSalidaBodega()

2. ComponenteAutorizacion (Recursos Humanos)

Este componente se encarga de gestionar la autorización de los vendedores dentro del sistema, garantizando que únicamente aquellos habilitados puedan realizar operaciones.

Funcionalidades:

- autorizarVendedor() → Permite habilitar un vendedor para operar en el sistema.
- revocarAutorizacion() → Permite deshabilitar un vendedor.

Relación con el sistema:

Este componente es fundamental, ya que controla el acceso al resto de funcionalidades. Es consumido por los clientes antes de permitir la ejecución de operaciones de venta.

3. ComponenteRegistroVentas (Ventas)

Encargado de gestionar el proceso de ventas, incluyendo la validación del vendedor, la selección de productos y el cálculo del total de la transacción.

Funcionalidades:

- registrarVenta() → Registra una venta en el sistema.
- calcularTotal() → Calcula el valor total de la venta.
- getDetalle() → Obtiene el detalle de la transacción.

Relación con el sistema:

Este componente depende de:

- ComponenteAutorizacion → para validar vendedores
- ComponenteInventario → para verificar disponibilidad

4. ComponenteInventario (Bodega)

Gestiona el control de stock de los productos disponibles en la bodega, así como los movimientos de entrada y salida.

Funcionalidades:

- verificarDisponibilidad() → Consulta la cantidad disponible de un producto.
- registrarEntrada() → Registra ingreso de productos al inventario.
- registrarSalida() → Registra salida de productos del inventario.

Relación con el sistema:

Este componente es utilizado por:

- Ventas → para validar stock antes de vender
- Entregas → para descontar productos
- Reabastecimiento → para detectar faltantes

5. ComponenteReabastecimiento (Bodega / Proveedores)

Se encarga de gestionar la reposición de productos cuando el inventario es insuficiente.

Funcionalidades:

- solicitarReabastecimiento() → Genera solicitud de productos a proveedores.

Relación con el sistema:

Este componente interactúa con:

- ComponenteInventario → para identificar necesidad de stock
- Proveedor → para gestionar pedidos

6. ComponenteGestionEntregas (Entregas)

Administra el proceso logístico de entrega de productos al cliente final.

Funcionalidades:

- registrarEntrega() → Registra una entrega en el sistema.

- confirmarEntrega() → Confirma la entrega realizada.
- consultarPedidosPendientes() → Lista entregas pendientes.
- registrarSalidaBodega() → Registra salida de productos desde bodega.

Relación con el sistema:

Este componente interactúa con:

- Ventas → para obtener pedidos
- Inventario → para actualizar stock

Integración de componentes

Los componentes no funcionan de manera aislada, sino que se integran entre sí para cumplir los procesos del sistema:

- Recursos Humanos habilita vendedores
- Ventas procesa transacciones
- Bodega controla inventario
- Proveedores abastecen productos
- Entregas distribuye pedidos

Este modelo permite que cada componente sea reutilizable, facilitando su consumo desde diferentes clientes sin necesidad de modificar la lógica interna.

La implementación basada en componentes permite desacoplar la lógica del sistema en módulos independientes, facilitando la reutilización de software, la escalabilidad de la solución y la integración con múltiples clientes, lo cual es un objetivo central de esta actividad.

Parte 5. Implementación De Requisitos Funcionales

A continuación, se presenta la implementación de los cinco requisitos funcionales extraídos del problema de PoliMarket, desarrollada en Node.js con una arquitectura basada en componentes. El proyecto incluye dos clientes en plataformas diferentes.

Estructura del proyecto

- db.js - Base de datos en memoria (vendedores, clientes, productos, proveedores, ventas, entregas)
- components/ComponenteAutorizacion.js - RF1
- components/ComponenteRegistroVentas.js - RF2
- components/ComponenteInventario.js - RF3
- components/ComponenteReabastecimiento.js - RF4
- components/ComponenteGestionEntregas.js - RF5
- clients/web/servidor.js - Cliente 1: API REST con Express (puerto 3000)
- clients/cli/cliente-cli.js - Cliente 2: CLI interactivo por terminal

RF1 - Autorizar vendedor

Descripción:

Permite que el área de Recursos Humanos autorice a un vendedor para que pueda interactuar con el sistema.

Componente asociado:

- ComponenteAutorizacion

Funcionamiento:

El sistema valida el identificador del vendedor y actualiza su estado a autorizado, permitiéndole acceder a las funcionalidades de ventas.

Importancia:

Este requisito es fundamental, ya que actúa como mecanismo de control de acceso, asegurando que solo usuarios autorizados puedan operar dentro del sistema.

RF2 - Registrar venta

Descripción:

Permite a un vendedor autorizado registrar una venta asociada a un cliente y uno o varios productos.

Componentes asociados:

- ComponenteRegistroVentas
- ComponenteAutorizacion
- ComponenteInventario

Funcionamiento:

- Se valida que el vendedor esté autorizado.
- Se verifica la disponibilidad de los productos en inventario.
- Se calcula el total de la venta.
- Se registra la transacción en el sistema.

Importancia:

Este requisito representa el núcleo del sistema, integrando múltiples componentes y asegurando la correcta ejecución del proceso de ventas.

RF3 - Verificar disponibilidad de productos

Descripción:

Permite consultar la cantidad disponible de un producto en la bodega antes de realizar una venta.

Componente asociado:

- ComponenteInventario

Funcionamiento:

El sistema consulta el stock del producto solicitado y retorna la cantidad disponible, permitiendo tomar decisiones antes de procesar la venta.

Importancia:

Evita inconsistencias en el sistema, como ventas de productos sin disponibilidad, garantizando la integridad del inventario.

RF4 - Solicitar reabastecimiento

Descripción:

Permite generar solicitudes de reposición de productos cuando el inventario alcanza niveles bajos.

Componente asociado:

- ComponenteReabastecimiento

Funcionamiento:

Cuando se detecta un nivel bajo de stock, el sistema genera automáticamente una solicitud hacia los proveedores para reponer los productos necesarios.

Importancia:

Garantiza la continuidad del negocio, evitando la falta de productos disponibles para la venta.

RF5 - Registrar entrega

Descripción:

Permite gestionar la entrega de productos al cliente y actualizar el estado de la venta.

Componentes asociados:

- ComponenteGestionEntregas
- ComponenteInventario

Funcionamiento:

- Se identifican los pedidos pendientes.
- Se registra la entrega correspondiente.
- Se descuenta el stock del inventario.
- Se actualiza el estado de la venta.

Importancia:

Permite completar el ciclo de venta, asegurando la correcta distribución de los productos y el control del inventario.

Integración con los clientes

Los requisitos funcionales implementados son consumidos por dos tipos de clientes:

- Cliente Web (API REST con Express):
- Permite acceder a las funcionalidades mediante peticiones HTTP.
- Cliente CLI (línea de comandos):
- Permite interactuar con el sistema desde la terminal.

Ambos clientes consumen los mismos componentes, evidenciando la reutilización de software y el desacoplamiento entre la lógica de negocio y la interfaz de usuario. La implementación de los requisitos funcionales mediante componentes reutilizables demuestra la efectividad del enfoque modular, permitiendo que múltiples clientes accedan a la misma lógica de negocio sin duplicación de código, lo cual constituye un principio fundamental en el desarrollo de software moderno.

Parte 6. Diagrama de componentes del sistema PoliMarket

A continuación, se presenta tanto el código como el diagrama de los componentes identificados para el sistema PoliMarket, junto con sus atributos, métodos y justificación de inclusión. El diagrama fue generado con PlantUML usando la sintaxis UML 2.5.

1. Código:

File Edit Selection View Go Run Terminal Help

EXPLORER

LABORATORIO NO. 2

- polimarket
  - {models,components,clients
  - clients
    - cli
      - cliente-cli.js
    - web
      - servidor.js
  - components
    - ComponenteAutorizacion.js
    - ComponenteGestionEntregas.js
    - ComponenteInventario.js
    - ComponenteReabastecimiento.js
    - ComponenteRegistroVentas.js
  - node_modules
  - public
  - uml
    - diagrama_clases.bmp
    - diagrama-clases.puml
    - diagrama-componentes.BMP
    - diagrama-componentes.puml
  - db.js
  - package-lock.json
  - package.json
- ~$liMarket_Unidad2_Alejandro_De_Mendoza_Final.docx
- ~$sarrollo Proyecto Alejandro De Mendoza.docx
- ~$tividad Evaluativa_Sumativa_U2.docx
- ~WRL1168.tmp
- Actividad Evaluativa_Sumativa_U2.docx
- Actividad Evaluativa_Sumativa_U2.pdf
- Desarrollo Proyecto Alejandro De Mendoza.docx
- PoliMarket_Unidad2_Alejandro_De_Mendoza_Final.docx
- PoliMarket_Unidad2_Alejandro_De_Mendoza_Final.pdf

diagrama-clases.puml | diagrama-componentes.puml

polimarket > uml > diagrama-componentes.puml > { } PoliMarket_Componentes

```
@startuml PoliMarket_Componentes

skinparam componentStyle uml2
skinparam backgroundColor #FFFFFF
skinparam componentBackgroundColor #FAFAFA
skinparam componentBorderColor #555555
skinparam arrowColor #333333
skinparam shadowing false
skinparam linetype ortho
skinparam defaultFontSize 12

' ==========================================
' CLIENTES
' ==========================================
package "Clientes" {
  [Cliente Web (API REST)]
  [Cliente CLI]
}

' ==========================================
' RECURSOS HUMANOS
' ==========================================
package "Recursos Humanos" {
  [ComponenteAutorizacion]
  [ComponenteGestionVendedores]
}

' ==========================================
' VENTAS
' ==========================================
package "Ventas" {
  [ComponenteRegistroVentas]
  [ComponenteConsultaVentas]
}

' ==========================================
' BODEGA
' ==========================================
package "Bodega" {
  [ComponenteInventario]
  [ComponenteReabastecimiento]
}

' ==========================================
' PROVEEDORES
' ==========================================
package "Proveedores" {
  [ComponenteGestionProveedores]
  [ComponentePedidos]
}

' ==========================================
' ENTREGAS
' ==========================================
package "Entregas" {
  [ComponenteGestionEntregas]
  [ComponenteLogistica]
}

' ==========================================
' RELACIONES CLIENTES
' ==========================================
[Cliente Web (API REST)] --> [ComponenteRegistroVentas]
[Cliente Web (API REST)] --> [ComponenteAutorizacion]
[Cliente CLI] --> [ComponenteRegistroVentas]
[Cliente CLI] --> [ComponenteInventario]

' ==========================================
' RELACIONES ENTRE COMPONENTES
' ==========================================

' RRHH
[ComponenteAutorizacion] --> [ComponenteGestionVendedores]

' Ventas
[ComponenteRegistroVentas] --> [ComponenteConsultaVentas]
[ComponenteRegistroVentas] --> [ComponenteAutorizacion]
[ComponenteRegistroVentas] --> [ComponenteInventario]

' Bodega
[ComponenteInventario] --> [ComponenteReabastecimiento]

' Proveedores
[ComponenteReabastecimiento] --> [ComponentePedidos]
[ComponenteGestionProveedores] --> [ComponenteInventario]

' Entregas
[ComponenteGestionEntregas] --> [ComponenteLogistica]
[ComponenteLogistica] --> [ComponenteInventario]
[ComponenteLogistica] --> [ComponenteConsultaVentas]

@enduml
```

2. Imagen del diagrama en PlantUML

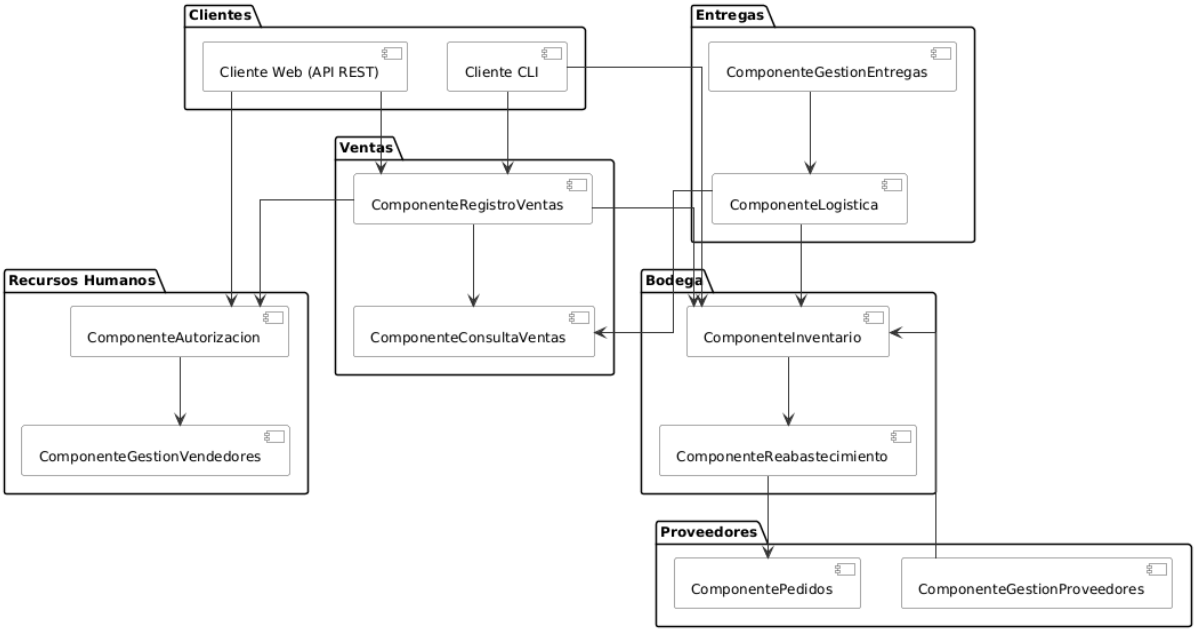


Figura 2. Diagrama de componentes del sistema PoliMarket

La imagen anterior corresponde al diagrama de componentes del sistema PoliMarket, exportado en formato BMP desde la herramienta PlantUML (plantuml.com) utilizando la sintaxis UML 2.5. El diagrama evidencia los diez componentes distribuidos por área de negocio y las relaciones de dependencia entre ellos, mostrando cómo interactúan para proveer las funcionalidades de la organización.

3. Imagen del diagrama en Visual Code

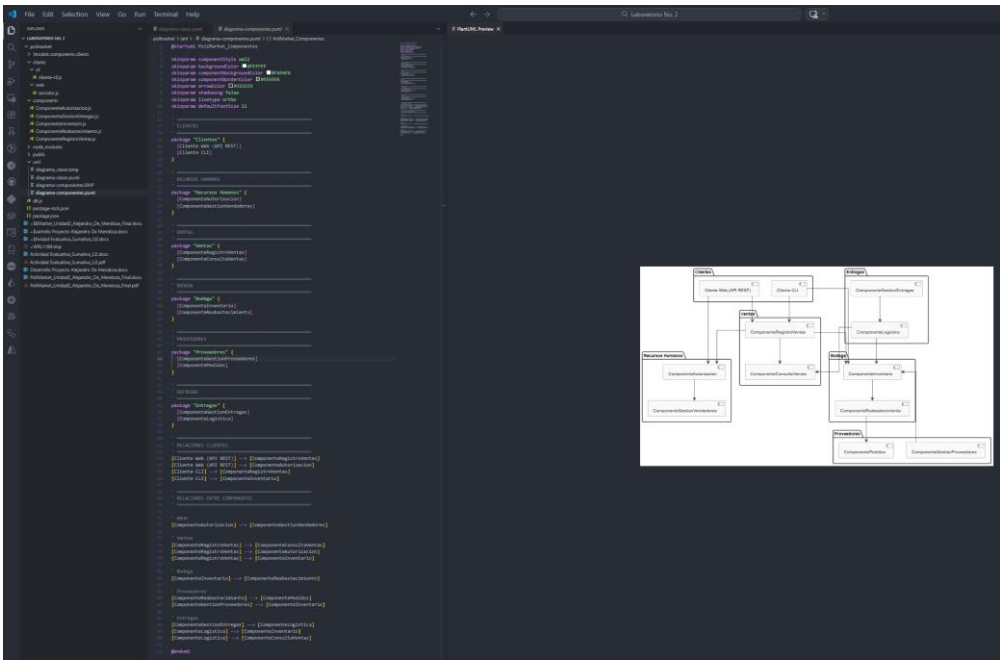


Figura 3. Diagrama de componentes del sistema PoliMarket en Visual Code

Parte 7. Clientes implementados

1. Cliente 1 - API REST con Express (plataforma web)

Expone todos los componentes como endpoints HTTP. Permite consumir los cinco RF a través de peticiones POST y GET. Se ejecuta en el puerto 3000 e incluye una interfaz web en http://localhost:3000.

- POST /rrhh/autorizar - RF1: Autorizar vendedor
- POST /ventas/registrar - RF2: Registrar venta
- GET /bodega/disponibilidad/:id - RF3: Verificar disponibilidad
- POST /bodega/reabastecer - RF4: Solicitar reabastecimiento
- POST /entregas/registrar - RF5: Registrar entrega

2. Cliente 2 - CLI interactivo (plataforma terminal)

Interfaz de línea de comandos que permite al usuario interactuar con el sistema a través de un menú numerado. Consume los cinco RF de forma interactiva sin necesidad de un servidor HTTP.

1. Autorizar vendedor (RF1 - RRHH)
2. Registrar venta (RF2 - Ventas)
3. Ver disponibilidad bodega (RF3 - Bodega)
4. Solicitar reabastecimiento (RF4 - Bodega)
5. Registrar entrega (RF5 - Entregas)

3. Tabla de funcionalidades por cliente

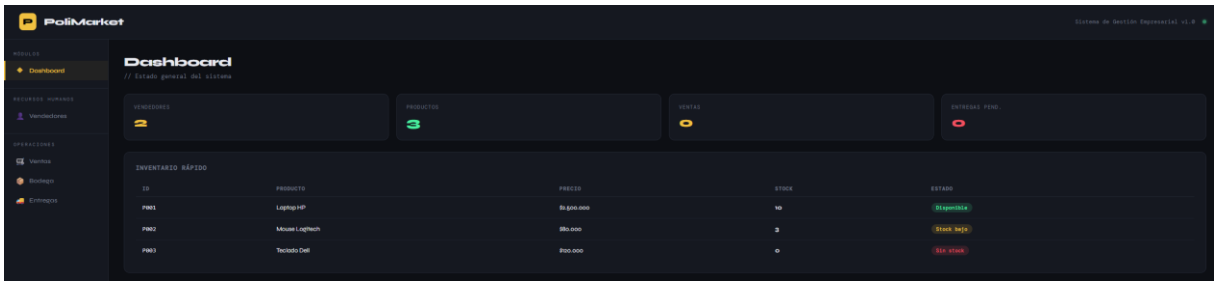
Cliente	Plataforma	Funcionalidades consumidas
Cliente Web (API REST)	HTTP / Navegador	RF1 autorizarVendedor(), RF2 registrarVenta(), RF3 verificarDisponibilidad(), RF4 solicitarReabastecimiento(), RF5 registrarEntrega()
Cliente CLI	Terminal	RF1 autorizarVendedor(), RF2 registrarVenta(), RF3 verificarDisponibilidad(), RF4 solicitarReabastecimiento(), RF5 registrarEntrega()

Ambos clientes consumen los cinco requisitos funcionales implementados, superando el mínimo de dos funcionalidades por cliente exigido en las condiciones de entrega.

Parte 8. Instrucciones de ejecución del programa

- Descomprimir el archivo polimarket.zip
- Ejecutar: npm install
- Cliente web: npm run start:web - acceder desde http://localhost:3000
- Cliente CLI: cd polimarket && npm run start:cli

Parte 9. Imagen Del Desarrollo



La imagen muestra el dashboard del sistema PoliMarket ejecutándose localmente en http://localhost:3000. Se evidencia la interfaz web construida como Cliente 1 (API REST con Express), donde se visualizan los módulos de gestión: inventario rápido con estado de stock por producto, contadores de vendedores, productos, ventas y entregas pendientes. Esta interfaz consume directamente los componentes implementados en Node.js.

Parte 10. Archivos Adjuntos En Adición

Es importante para finalizar este desarrollo que en adición a la entrega se adjuntan los siguientes archivos de complemento para análisis y ejecución:

- diagrama-clases.bmp
- diagrama-componentes.bmp
- polimarket.zip (código fuente completo)

Y con este último punto doy por finalizada la entrega de esta actividad, mil gracias respetada profesora por los conocimientos transmitidos.

CONCLUSIONES DEL DESARROLLO

Para concluir el presente trabajo, puede afirmarse que el desarrollo del modelo de clases UML permitió establecer una base sólida para la comprensión y estructuración del sistema de información propuesto para la empresa PoliMarket. La identificación de las clases candidatas, junto con la definición de sus atributos, métodos y responsabilidades, facilitó la construcción de una representación clara y coherente de los procesos de negocio involucrados, tales como la gestión de ventas, el control de inventario, la autorización de vendedores y la distribución de productos.

Desde la perspectiva del diseño de software, la aplicación de los principios del enfoque orientado a objetos permitió estructurar el sistema de manera modular, promoviendo buenas prácticas como la separación de responsabilidades, la reutilización de código y el bajo acoplamiento entre componentes. Asimismo, la organización del sistema en áreas de negocio como recursos humanos, ventas, bodega, proveedores y entregas evidenció una adecuada descomposición del problema, permitiendo una mejor comprensión de la arquitectura del sistema.

Por otra parte, la definición de las relaciones entre las clases permitió representar de forma precisa las interacciones del sistema. El uso de asociaciones, dependencias y

multiplicidades contribuye a una mejor organización del modelo, facilitando la comprensión de cómo los diferentes elementos se conectan y colaboran entre sí. Esto resulta fundamental para garantizar un diseño coherente y alineado con los requerimientos planteados en el contexto organizacional de PoliMarket.

En este sentido, la incorporación de un enfoque basado en componentes permitió estructurar la lógica del sistema en módulos independientes, cada uno responsable de una funcionalidad específica. Este enfoque facilita la reutilización de software, ya que los componentes pueden ser consumidos por diferentes clientes sin necesidad de modificar su implementación, lo cual se evidencia en el desarrollo de la API REST y el cliente CLI.

Asimismo, la implementación de los cinco requisitos funcionales utilizando Node.js permitió materializar el diseño propuesto, demostrando cómo el modelo conceptual puede ser llevado a una solución funcional. La interacción de los clientes con los componentes desarrollados evidencia el desacoplamiento entre la lógica de negocio y las interfaces de usuario, lo cual es un principio clave en el desarrollo de software moderno.

En síntesis, el desarrollo de esta actividad demuestra la importancia del modelado y la reutilización de software como elementos fundamentales dentro del ciclo de vida del desarrollo. La construcción de modelos UML bien definidos y la implementación basada en componentes contribuyen significativamente a mejorar la calidad, escalabilidad y mantenibilidad de los sistemas.

Finalmente, este ejercicio permite evidenciar cómo el uso de UML, el enfoque orientado a objetos y la arquitectura basada en componentes proporcionan un marco sólido para el análisis, diseño e implementación de sistemas empresariales, fortaleciendo las competencias necesarias para abordar de manera estructurada el desarrollo de soluciones tecnológicas en contextos reales.

BIBLIOGRAFÍA

A continuación, se presenta la bibliografía utilizada como base conceptual y metodológica para el desarrollo del presente proyecto:

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). El lenguaje unificado de modelado (UML): Guía del usuario (2ª ed.). Pearson Educación.
- Sommerville, I. (2016). Ingeniería de software (10ª ed.). Pearson Educación.
- Pressman, R. S., & Maxim, B. R. (2020). Ingeniería de software: Un enfoque práctico (9ª ed.). McGraw-Hill.
- Szyperski, C. (2002). Component Software: Beyond Object-Oriented Programming (2nd ed.). Addison-Wesley.
- Object Management Group (OMG). (2017). Unified Modeling Language (UML) Specification Version 2.5. Recuperado de: https://www.omg.org/spec/UML/
- PlantUML. (2024). PlantUML Language Reference Guide. Recuperado de: https://plantuml.com/
- Node.js Foundation. (2024). Node.js Documentation. Recuperado de: https://nodejs.org/
- Express.js. (2024). Express - Node.js web application framework. Recuperado de: https://expressjs.com/

AGRADECIMIENTO

Expreso mi más sincero agradecimiento a la Ingeniera Lina María Montoya Suárez, por su orientación, acompañamiento y compromiso durante el desarrollo de la presente actividad. Su guía ha sido fundamental para fortalecer la comprensión de los conceptos relacionados con el modelado de software, la reutilización de componentes y la aplicación del diseño orientado a objetos en contextos reales.

Asimismo, agradezco el espacio académico brindado, el cual permitió aplicar de manera práctica los conocimientos adquiridos, no solo en la construcción de modelos UML, sino también en la estructuración de arquitecturas basadas en componentes y en la implementación de soluciones funcionales mediante herramientas tecnológicas como Node.js.

Finalmente, valoro el acompañamiento constante y las herramientas proporcionadas, las cuales contribuyeron significativamente al desarrollo de este trabajo y al fortalecimiento de mis competencias en el análisis, diseño e implementación de sistemas de software, desde una perspectiva estructurada y orientada a la reutilización.

¡¡¡Mil gracias profesora!!!

Respetuosamente,

ALEJANDRO DE MENDOZA
Ingeniero Informático
Ingeniero de Plataformas e Integraciones